Driving a DAC with FreeRTOS ticks and using HAL

Big picture of this lab

- You will use an existing DAC driver to create a sawtooth wave and view it on a scope.
- The timing for the sawtooth comes from the FreeRTOS tick. The DAC driver matches the Arduino API, and is implemented with individual CSR writes. You won't touch the driver, but will flesh out *lab4_main.c* to use the driver and create your sawtooth.

What you will learn

- This is your first lab using a DAC. You'll get experience with DACs, and learn to program them with CSRs.
- You will see the limitations of using FreeRTOS to drive high speed signals.
- You will get more practice debugging FreeRTOS systems.

Part one: use FreeRTOS and an existing DAC driver to create a sawtooth wave and view it on the scope

Start by building a FreeRTOS project as usual. Then add library code files *lib_clocks.c, lib_GPIO.c, lib_DAC.c*, the main code *lab4_main.c*, and the usual *lib_ee152.h*. As always, all .c files go into the *src* directory and all .h files into *include*.

Look at *lib_DAC.c*. You will see that it implements the Arduino interface *analogWrite*(*pin*, *value*). Take a look at *lib_DAC.c* to see how *analogWrite* works, comparing it to our discussion of the STM32 DAC architecture from class.

Next, look at *lab4_main.c*. You will see that you have a basic framework to set up FreeRTOS and create two tasks:

- task_DAC_sawtooth(): use the DAC driver *analogWrite*() to drive a sawtooth waveform onto the Nano pin A3.
- task_blinky_green(): blink the green LED on and off at two blinks per second.

Your job: first flesh out the calls to instantiate these two functions (using appropriate priorities). Second, write them! You should create a sawtooth with a frequency of 100Hz and an amplitude of reasonably close to 3.3V, using *analogWrite*(). You should not use any spin-loop delays.

When you're done, show your code to a TA, as well as displaying the scope waveforms.

Part two: rewrite the DAC driver using HAL

You've been given a perfectly serviceable DAC driver that works by writing individual CSR bits. Your job for part two of this lab is to rewrite the driver using HAL.

You'll notice that *lib_DAC.c* already has an area for you to write your new code, delimited by the lines

```
#ifdef USE_HAL
…
#endif
```

That's where you will put your driver. You should not need to touch *lib_ee152.h*. You should finish writing *analogWrite*(), as well as adding any other code you need. The idea is that by rewriting *analogWrite*() in HAL, you need not touch *lab4_main.c*.

In order for the compiler to find all of the HAL library files that you will need, you must edit *platformio.ini* and change the line

```
framework = cmsis
```

to

```
framework = stm32cube
```

This gets PlatformIO to change its include paths so as to access all of the .c and .h files that make up HAL (which PlatformIO downloaded and saved away when you first installed PlatformIO).

Furthermore, to make our own library code work with HAL, you must add the line

```
-DUSE_HAL
```

to *platformiol.ini*; this gets *lib_DAC.c* (and a few other of our files) to compile the correct code.

OK, now you know what to do – but how do you do it? How do you figure out how to write the HAL code? You have several choices:

- You can use CubeMX just as we did for the GPIO and as demoed in class. You'll have to take a dozen or so lines of code from the *main.c* that CubeMX created, and cut/paste it into *lib_DAC.c*. However, CubeMX will only write the DAC setup code, not the code that sets new DAC values.
- You can directly use the HAL manual, which is on the course website as usual.
- There are plenty of HAL examples online that you are welcome to try.
- Copilot can often do a pretty good job.

However you choose to write your code, you still most likely won't get it completely correct on the first try. So the next problem is how to debug your code.

Debugging this type of code is a classic example of "not always easy!" First, you are using the HAL API and you probably don't fully understand how to use it. Second, you're back to the usual FreeRTOS issue of how to debug code that switches processes 1000 times per second. Here are two hints to keep your sanity:

- At the end of the day, HAL is still responsible for setting the same CSR bits that our original driver did. Thus, you have a set of known-good CSR settings that you can compare against. Furthermore, platformIO makes it very easy to examine CSR settings. So you can execute your HAL setup, look at the appropriate CSR settings and compare them against your known-good version. This should help you narrow down any problems.
- Of course, it's hard to examine CSR settings when processes are being swapped in and out by FreeRTOS. You can make your life simpler by initially debugging without FreeRTOS. So, e.g., you can rewrite *lab4_main.c* to call *task_DAC_sawtooth*() directly from *main*() without ever starting the FreeRTOS scheduler. Of course, you will have to replace the *vTaskDelay*() call inside of *task_DAC_sawtooth*() with a spin-loop wait like we did in lab #1.

Checkout

- Demonstrate to one of us that your code works. Use a scope to show the sawtooth waveform in both cases. Make sure to show it working on both pins A3 and A4 (i.e., on both DAC1 and DAC2).
- Your waveform probably has a "staircase" shape rather than a true sawtooth. Can you explain why, and also explain the number of steps in the staircase?
- Walk through the CSR settings using the PlatformIO debugger to show a few of the bits that are set and explain what they do. Just a few of the bits is fine.

What to turn in

Turn in a report that includes the relevant code snippets (*task_sawtooth*, *task_blink_green* and their priorities; and the HAL version of *analogWrite*), and answers the following questions:

1. By the time we use our version of the Arduino API, we've added substantial overhead to *analogWrite*(). Can you list some of the overhead?
2. Using the FreeRTOS tick to time our sawtooth was nice and simple. However, we were fairly limited in the waveforms we could create. If we tried to create musical

notes (and say we wanted them to roughly be sine waves), what are some limits on the note frequencies that we could reasonably build?

3. Many groups had sawtooths with a fair amount of jitter on the scope. Explain a code scenario in *task_DAC_sawtooth*() that could cause the problem, and how to fix it.